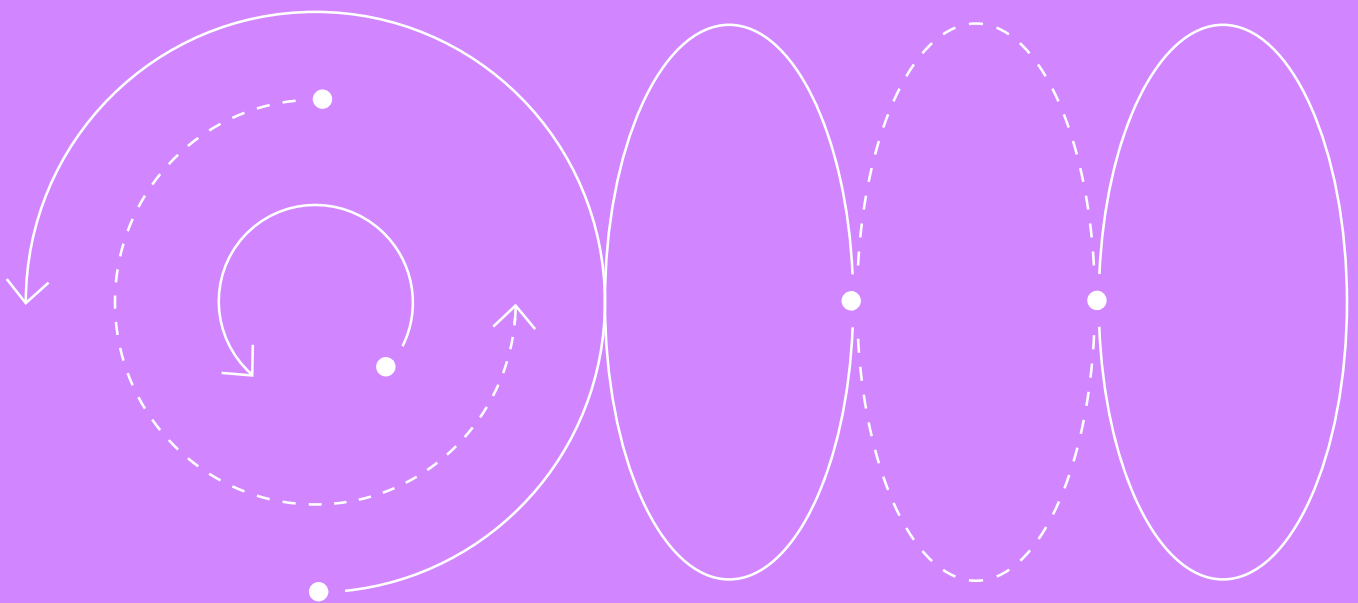


Оптимизация запросов. План запроса



SQL И БАЗЫ ДАННЫХ (ДЛЯ РАЗРАБОТЧИКОВ)

ЛОНГРИД

НЕДЕЛЯ 8

Введение

Оптимизация SQL-запросов — это процесс улучшения производительности, который требует глубокого понимания как структуры данных, так и особенностей работы СУБД. Оптимизация играет ключевую роль в обеспечении высокой производительности баз данных. При работе с данными корректная оптимизация позволяет следующее.

1. **Сократить время выполнения запросов** — уменьшение задержек повышает общую производительность и скорость выполнения конкретно твоего запроса.
2. **Снизить нагрузку на сервер** — уменьшение потребления CPU и памяти.
3. **Повысить масштабируемость** — оптимизированные запросы позволяют обрабатывать больше данных с меньшими затратами.

Для понимания оптимизации, необходимо разобраться в особенностях и ключевых понятиях:

- планировщик запросов,
- операции планировщика запросов,
- команды **EXPLAIN** и **EXPLAIN ANALYZE**.

Планировщик запросов

ТЕРМИН

Планировщик запросов (query planner) — это компонент СУБД, который анализирует SQL-запрос, строит возможные планы его выполнения и выбирает наилучший из них на основе стоимости выполнения.

Основные этапы работы планировщика

1. **Разбор SQL-запроса (parsing)**
Синтаксический и семантический анализ запроса.
2. **Анализ и переписывание (rewriting)**
Применение правил оптимизации, преобразование представлений в подзапросы и упрощение выражений.
3. **Генерация возможных планов (planning)**
Создание множества альтернативных стратегий выполнения.
4. **Оценка стоимости (cost estimation)**
Вычисление затрат на выполнение каждого плана с учётом статистики о данных.
5. **Выбор оптимального плана (plan selection)**
Выбор наиболее эффективного варианта на основе минимальной стоимости.

ФАКТОРЫ, ВЛИЯЮЩИЕ НА ВЫБОР ПЛАНА

Планировщик использует статистику о данных и индексах для принятия решений. Основные факторы:

- **параллельное выполнение (parallel execution)** — в аналитических СУБД нагрузка может распределяться между узлами кластера;
- **кеширование и буферизация** — ранее выполненные запросы могут повторно использовать кешированные результаты;
- **объём данных в таблицах** — чем больше строк, тем важнее эффективные индексы и разбиение;
- **типы соединений** — джойны имеют разную стоимость в зависимости от данных;
- **наличие индексов** — использование индексов влияет на стоимость выполнения запроса;
- **партиционирование таблиц** — позволяет избегать сканирования ненужных разделов данных.

ТЕРМИН
План запроса — это наиболее эффективный вариант запроса по версии планировщика. Представляет собой последовательность операций в виде дерева, которые выполняет БД для получения требуемого в запросе результата. Построением плана запроса занимается оптимизатор .

В PostgreSQL фактический план выполнения запроса можно увидеть с помощью команд **EXPLAIN** и **EXPLAIN ANALYZE**.

Пример выполнения команды **EXPLAIN**

QUERY PLAN	
1	Sort (cost=28.88..29.32 rows=175 width=128)
2	Sort Key: n.nspname, c.relname
3	-> Hash Join (cost=1.18..22.36 rows=175 width=128)
4	Hash Cond: (c.relnamespace = n.oid)
5	-> Seq Scan on pg_class c (cost=0.00..20.48 rows=175 width=68)
6	Filter: (relkind = 'i'::"char")
7	-> Hash (cost=1.08..1.08 rows=8 width=68)
8	-> Seq Scan on pg_namespace n (cost=0.00..1.08 rows=8 width=68)

Как видно по плану, SQL-оптимизатор использует незнакомые ранее операции. Сперва необходимо познакомиться с основными операциями и далее научиться читать план запроса.

Основные операции в плане запроса

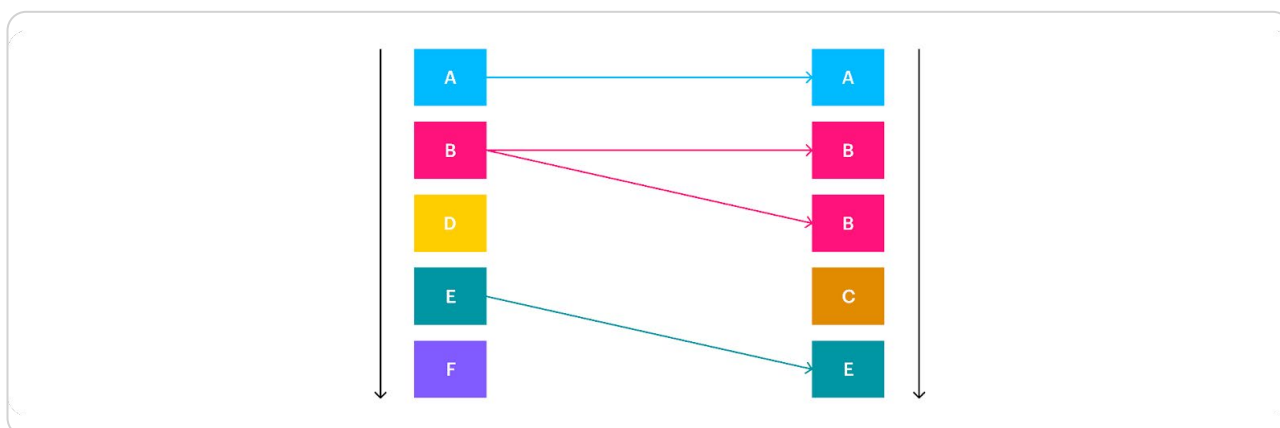
УЗЛЫ ЧТЕНИЯ ДАННЫХ

Scan-узлы отвечают за чтение строк из таблиц или индексов. Выбор типа сканирования (последовательный, по индексу, bitmap-скан или TID-скан) оказывает существенное влияние на производительность запросов.

- **Seq Scan** — полное сканирование таблицы. Используется, если нет подходящих индексов и ожидается выборка большей части строк.
- **Index Scan / Index Only Scan** — построчное чтение через индекс.
- **Bitmap Index Scan / Bitmap Heap Scan** — сначала строится битовая карта нужных страниц, затем они читаются.
- **Tid Scan / Tid Range Scan** — выборка по физическим позициям строк.

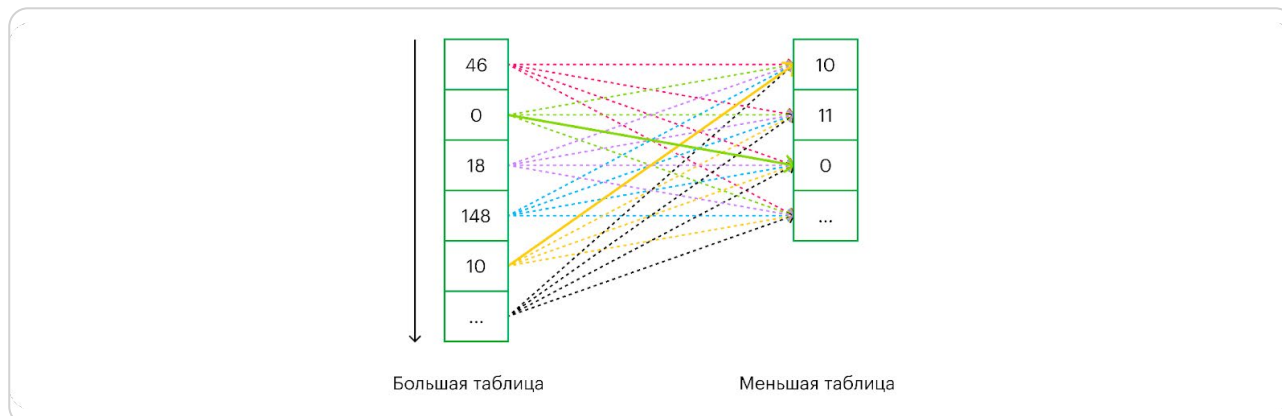
УЗЛЫ СОЕДИНЕНИЯ ДАННЫХ

1. **Merge Join** — соединение предварительно отсортированных таблиц. Происходит за один проход по каждой из таблиц.



Для вычисления Merge Join необходима предварительная **сортировка** данных, а сортировка — очень **тяжёлая** операция. Merge Join встречается очень редко, и если он встретился, то, скорее всего, ты что-то делаешь не так.

2. **Nested Loop:** данное соединение можно интерпретировать как построчный проход по обеим таблицам, что является **крайне неоптимальным** способом соединения в случае, когда обе таблицы **большие**. Этот способ соединения выбирается оптимизатором при «плохом» условии соединения (отличном от обычного равенства полей).

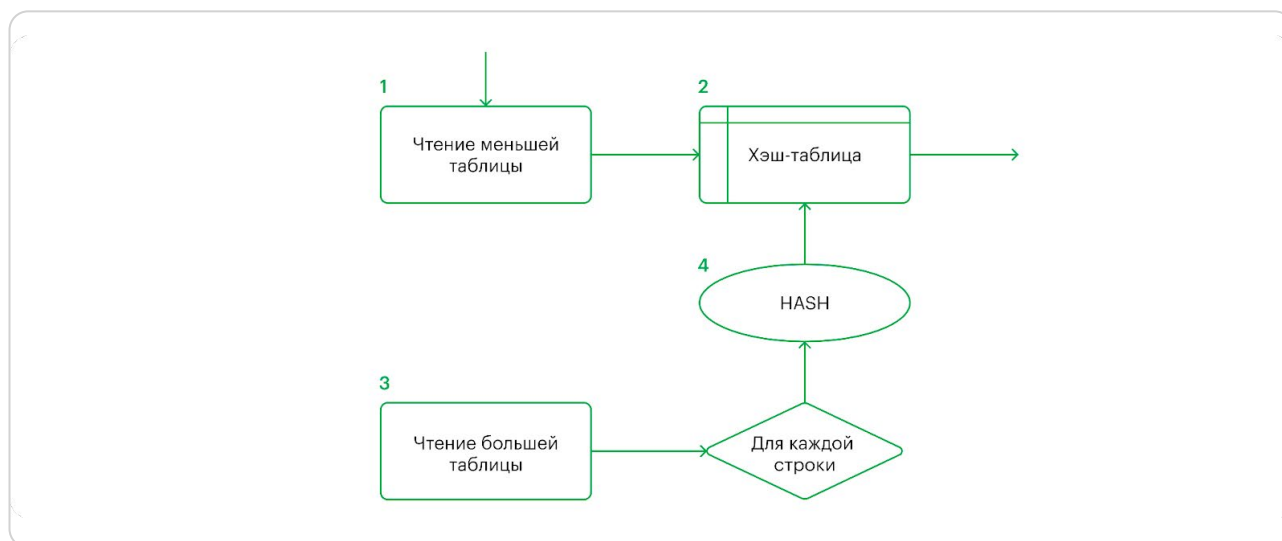


Соединение двух таблиц с помощью Nested Loop выполняется следующим образом.

1. Выполняется проход по строкам большей таблицы.
2. Для каждой строки выполняется проход по всем строкам меньшей таблицы.
3. В случае выполнения условия соединения соответствующая пара строк попадает в результирующую таблицу.

Таким образом, меньшая таблица просматривается столько раз, сколько строк в большей таблице. Общее количество проверок равно количеству строк большей таблицы, умноженному на количество строк меньшей таблицы. Для выполнения Nested Loop меньшая таблица **копируется** (Broadcast Motion) на все сегменты. При этом меньшая таблица не всегда является маленькой.

3. **Hash Join** — наиболее часто встречающийся тип соединения таблиц и в подавляющем большинстве наиболее оптимальный из всех.



Соединение двух таблиц с помощью Hash Join выполняется следующим образом.

1. Происходит чтение меньшей таблицы.
2. На основе этой таблицы строится хеш-таблица — это специальная таблица, по которой можно очень быстро искать нужные значения.
3. Затем происходит чтение большей таблицы.
4. Для каждой строки большей таблицы вычисляется значение хеш-функции от ключа соединения и выполняется поиск этого значения в хеш-таблице. В случае успеха соответствующие записи из обеих таблиц добавляются к результату соединения.

Наиболее предпочтительным и в большинстве случаев оптимальным вариантом является Hash Join. Merge Join встречается редко, а Nested Loop является дорогим соединением.

Однако бывают ситуации, в которых без построчного прохода по обеим таблицам не обойтись.

УЗЛЫ АГРЕГАЦИИ И ГРУППИРОВКИ

- Aggregate/HashAggregate/GroupAggregate — вычисляют агрегаты (**SUM**, **COUNT** и другие), различаются стратегией: хеш, сортировка или комбинированная.
- Group — простая группировка без агрегатных функций.
- WindowAgg — оконные функции (**ROW_NUMBER()**, **RANK()** и тому подобное).

УЗЛЫ СОРТИРОВКИ И ОГРАНИЧЕНИЙ

- Sort / Incremental Sort — полная или поэтапная сортировка.
- Limit — применение **LIMIT/OFFSET**.
- Filter — принимает на вход поток строк от нижележащих узлов (Seq Scan, Index Scan, Hash Join и тому подобное) и проверяет условие на каждой строке (предикат **WHERE**).

УЗЛЫ ПАРАЛЛЕЛИЗМА

- Gather / Gather Merge — сбор и объединение результатов от параллельных воркеров.
- Parallel Seq Scan, Parallel Index Scan и другие — распределение чтения по воркерам.

ВСПОМОГАТЕЛЬНЫЕ УЗЛЫ

- Materialize — кеширование промежуточного результата в памяти.
- Memoize — кеширование результатов выражения или подзапроса по ключу.
- Unique — удаление дубликатов.
- SetOp — реализация операций множеств (**UNION**, **INTERSECT**, **EXCEPT**).
- Partition Selection — это служебный узел, который определяет, какие партиции нужно читать, а какие можно отбросить.

ФУНКЦИИ УЗЛОВ

- Модульность: каждый узел решает отдельную задачу (чтение, фильтрация, соединение и так далее).
- Оптимизация: оптимизатор может менять порядок и метод конкретного узла для достижения максимальной по его мнению производительности.
- Диагностика: по дереву узлов проще понять, где в запросе находится узкое место по производительности.

Чтение плана запроса

EXPLAIN

Для того чтобы увидеть план запроса, нужно использовать команду **EXPLAIN** перед запросом.

С помощью данной команды мы увидим предполагаемый план выполнения запроса, который в редких случаях может отличаться от фактического. При этом сам запрос не будет запущен.

В выводе команды **EXPLAIN** каждая строка соответствует узлу дерева плана запроса. Чтение плана запроса выполняется от самого вложенного узла (часто им является самый правый нижний узел).

В строке, помимо типа выполняемой операции, содержатся следующие метрики.

- **cost** — стоимость выполнения операции, измеряется в единицах стоимости, которые являются произвольными (то есть относительными), но обычно означают выборку страниц на диске. Первое значение — начальная стоимость (стоимость обработки первой записи), второе значение — итоговая стоимость (стоимость обработки всех строк).
- **rows** — итоговое количество строк. С помощью данной метрики можно увидеть, насколько точно оптимизатор оценивает количество строк в используемых таблицах и промежуточных вычислениях.
- **width** — средний размер (в байтах) одной строки, возвращаемой данной операцией.

ПРИМЕР

```
explain
select
    n.nspname as schema_name,
    c.relname as index_name
from pg_class as c
    join pg_namespace as n
        on n.oid = c.relnamespace
where c.relkind = 'i'
order by schema_name, index_name;
```

	QUERY PLAN
1	Sort (cost=28.88..29.32 rows=175 width=128)
2	Sort Key: n.nspname, c.relname
3	-> Hash Join (cost=1.18..22.36 rows=175 width=128)
4	Hash Cond: (c.relnamespace = n.oid)
5	-> Seq Scan on pg_class c (cost=0.00..20.48 rows=175 width=68)
6	Filter: (relkind = 'i'::"char")
7	-> Hash (cost=1.08..1.08 rows=8 width=68)
8	-> Seq Scan on pg_namespace n (cost=0.00..1.08 rows=8 width=68)

Согласно плану, запрос выполняется следующим образом.

1. 8-я строка — чтение таблицы **pg_namespace** (оператор **Seq Scan**).
При этом оптимизатор считает, что в таблице 8 строк.
2. 7-я строка — на основе таблицы **pg_namespace** строится хеш-таблица (оператор **Hash**).
3. 5-я строка — чтение таблицы **pg_class** (оператор **Seq Scan**), в которой 175 строк.
4. 3-я строка — соединение таблиц (с помощью оператора **Hash Join**) по условию равенства полей **c.relnamespace = n.oid**
5. 1-я строка — сортировка (оператор **Sort**) по полям **n.nspname, c.relname**.

EXPLAIN ANALYZE

Отличие данной команды от предыдущей заключается в том, что, прежде чем вывести план, оптимизатор **выполняет** сам запрос, что позволяет вывести дополнительные метрики запроса, помимо указанных ранее.

Среди этих метрик наиболее полезны следующие.

- **Итоговое время** (в миллисекундах), которое работал запрос (строка 13).
- **actual time** — время в миллисекундах на каждую операцию (строки 11, 9, 6, 4, 1).
- **width** — средний размер (в байтах) одной строки, возвращаемой данной операцией
- **loops** — количество раз, которое данная операция была выполнена
- **Объем памяти** (width x rows), который потребовался во время операции.
- Прочая информация (строки 10, 8, 7, 5, 3, 2).

ПРИМЕР

```
explain analyze
select
    n.nspname as schema_name,
    c.relname as index_name
from pg_class as c
    join pg_namespace as n
        on n.oid = c.relnamespace
where c.relkind = 'i'
order by schema_name, index_name;
```

QUERY PLAN	
1	Sort (cost=29.24..29.69 rows=180 width=128) (actual time=0.364..0.380 rows=180 loops=1)
2	Sort Key: n.nspname, c.relname
3	Sort Method: quicksort Memory: 54kB
4	-> Hash Join (cost=1.18..22.50 rows=180 width=128) (actual time=0.046..0.233 rows=180 loops=1)
5	Hash Cond: (c.relnamespace = n.oid)
6	-> Seq Scan on pg_class c (cost=0.00..20.60 rows=180 width=68) (actual time=0.011..0.144 rows=180 loops=1)
7	Filter: (relkind = 'i':"char")
8	Rows Removed by Filter: 268
9	-> Hash (cost=1.08..1.08 rows=8 width=68) (actual time=0.025..0.026 rows=28 loops=1)
10	Buckets: 1024 Batches: 1 Memory Usage: 11kB
11	-> Seq Scan on pg_namespace n (cost=0.00..1.08 rows=8 width=68) (actual time=0.006..0.012 rows=28 loops=1)
12	Planning Time: 0.251 ms
13	Execution Time: 0.425 ms

Согласно плану видно, что все операции в точности совпадают с результатом выполнения команды **EXPLAIN**. Однако помимо ранее оговорённых метрик появились новые. Мы видим, сколько времени потратилось на весь запрос (строка 13) и на каждую операцию, а также фактическое количество обработанных строк, в строке 11 есть отличие между rows в первых скобках и rows во вторых. То есть оптимизатор «думал», что в таблице будет 8 строк, а на самом деле оказалось 28. Так происходит из-за неверной **статистики**.

СТАТИСТИКА

В общем случае существует множество вариантов для выполнения одного запроса и задача оптимизатора — выбрать среди них наиболее оптимальный. Для того чтобы выбрать наиболее оптимальный вариант, оптимизатор должен иметь некоторую информацию о таблицах, которые участвуют в запросе. Эта информация о таблицах называется **статистикой**. В статистике для таблицы содержится информация об объёме таблицы, для каждого поля таблицы содержится информация о количестве null-значений, уникальных и наиболее часто встречающихся значений и так далее.

В случае когда статистика на таблицу неактуальная, то есть после последнего сбора статистики данные в таблице сильно изменились, оптимизатор может выбирать неоптимальный план запроса. Для таких случаев имеется возможность принудительного сбора статистики на таблицу.

```
ANALYZE table_name;
```

Важно помнить, что операция сбора статистики является тяжёлой для больших таблиц. Для этих случаев имеется возможность собирать статистику только по требуемым полям.

```
ANALYZE table_name(field1[, field2, ...]);
```

В большинстве случаев достаточно собирать статистику только по полям, участвующим в условии соединения.

Для подробного вывода статистики можно запускать команду **ANALYZE** вместе с **VERBOSE**.

```
ANALYZE VERBOSE table_name;
```

```
analyzing "public.host"  
"host": scanned 1 of 1 pages, containing 5 live rows and 0 dead rows; 5 rows in sample, 5 estimated total rows  
analyzing "public.host"  
"host": scanned 1 of 1 pages, containing 5 live rows and 0 dead rows; 5 rows in sample, 5 estimated total rows  
analyzing "public.host"  
"host": scanned 1 of 1 pages, containing 5 live rows and 0 dead rows; 5 rows in sample, 5 estimated total rows
```

В подробном выводе можно заметить, что есть упоминание так называемых **dead rows** (мёртвые строки). При обычных **UPDATE/DELETE** старые версии строк не удаляются мгновенно, а остаются в таблицах и помечаются флагом **DELETED** для СУБД. Без очистки такие помеченные записи накапливаются, замедляют сканирование таблиц, увеличивая I/O-нагрузку, и ухудшают производительность запросов. Избавляться от «мёртвых» строк помогает команда **VACUUM**, которую можно вызвать принудительно для конкретной таблицы.

```
VACUUM table_name;
```

В PostgreSQL существует система **AUTOVACUUM**, которая в автоматическом режиме занимается удалением данных, а также сбором статистики.

AUTOVACUUM запускается, когда выполняется следующее условие.

```
autovacuum_analyze_threshold + (autovacuum_analyze_scale_factor  
× количество строк в таблице)
```

autovacuum_analyze_scale_factor по умолчанию равен 0,1.

autovacuum_analyze_threshold — 50.

Параметры конфигурации PostgreSQL

В PostgreSQL (как и в любой другой СУБД) существует множество параметров конфигурации, влияющих на производительность запросов.

ПАРАМЕТРЫ УПРАВЛЕНИЯ ПАМЯТЬЮ

- **shared_buffers** — объём памяти для буферного кеша PostgreSQL (используется для чтения/записи страниц). Если запрос выполняется слишком часто, его нужно перевести в буфер. Для этого стоит увеличить объём памяти для буфера.
- **effective_cache_size** — сколько памяти ОС выделит под файловый кеш, помимо **shared_buffers**.
- **work_mem** — память на одну операцию сортировки/хеширования в запросе (каждый узел плана может потребовать собственный кусок). При нехватке выделенной памяти PostgreSQL начинает использовать временные файлы на диске, что значительно замедляет выполнение запросов.
- **maintenance_work_mem** — как и **work_mem**, только для операций обслуживания (**CREATE INDEX**, **VACUUM**, **ALTER TABLE** и тому подобное).

ПАРАМЕТРЫ ПАРАЛЛЕЛИЗМА

- **max_worker_processes** — общее число вспомогательных процессов (включая параллельные запросы, репликацию, background workers). В принципе число таких процессов больше числа ядер процессора ставить нет смысла, а иногда и плохо. Проблемой при превышении числа процессов над количеством ядер является увеличение количества переключений контекста (context switching). Переключение контекста (context switching) — это механизм операционной системы, позволяющий разделять единичный физический процессор между несколькими процессами или потоками.
- **max_parallel_workers** — максимум воркеров для параллельных запросов в кластере. Та же рекомендация с числом ядер.

ПРОЧИЕ

- **autovacuum_naptime** — пауза между циклами autovacuum.
- **effective_io_concurrency** — максимальное число одновременных асинхронных операций чтения/записи, которое сессия может инициировать.
- **max_connections** — максимальное число подключений.

Изменяя эти параметры, можно добиться существенного прироста в производительности запросов, но важно быть предельно аккуратными.

Партиционирование

ТЕРМИН

Партиционирование (partitioning) — это способ повышения производительности запросов за счёт логического разбиения больших таблиц на небольшие подтаблицы, называемые партициями (partitions).

При выполнении запроса над партиционированной таблицей происходит сканирование ограниченного числа строк из общего набора данных путём обращения только к необходимым подтаблицам (partitions) на основе условий предикатов (where) вместо чтения всего набора данных. Принадлежность каждой новой записи таблицы к той или иной партиции определяется на основе значения **ключа партиционирования** (partition key) — столбца (или набора столбцов в случае многоуровневого партиционирования), который выбирается при создании таблицы. Партиционированная таблица представляет из себя набор подтаблиц, где каждая партиция — это отдельная таблица.



К столбцам, участвующим в партиционировании, запрещено применять операцию **update**!

ВИДЫ ПАРТИЦИОНИРОВАНИЯ

→ **by range** — по диапазону значений (чаще всего временному).

Таблица разбивается на диапазоны значений без пересечений между ними.

Границы диапазонов включают нижнее значение и исключают верхнее.

```
CREATE TABLE sales (  
    id BIGSERIAL,  
    sale_date DATE NOT NULL,  
    amount DECIMAL(12,2),  
    customer_id INT  
) PARTITION BY RANGE (sale_date);  
  
CREATE TABLE sales_2024_q1 PARTITION OF sales  
    FOR VALUES FROM ('2024-01-01') TO ('2024-04-01');  
CREATE TABLE sales_2024_q2 PARTITION OF sales  
    FOR VALUES FROM ('2024-04-01') TO ('2024-07-01');
```

→ **by list** — по значениям.

Данные распределяются на основе списка конкретных значений.

```
CREATE TABLE customers (  
    id SERIAL,  
    name TEXT,  
    region TEXT NOT NULL  
) PARTITION BY LIST (region);  
  
CREATE TABLE customers_north PARTITION OF customers  
    FOR VALUES IN ('North', 'Northeast');  
CREATE TABLE customers_south PARTITION OF customers  
    FOR VALUES IN ('South', 'Southeast');
```

→ **by hash** — по хешу.

Строки распределяются по физическим партициям на основе хеш-значения одного или нескольких столбцов-ключей. Каждая партиция получает параметры **MODULUS** (модуль) и **REMAINDER** (остаток), и в неё попадают все строки, для которых выполняется следующее условие.

→ $\text{hash(ключ)} \bmod \text{MODULUS} = \text{REMAINDER}$

```

CREATE TABLE orders (
    id          BIGSERIAL PRIMARY KEY,
    cust_id     INT NOT NULL,
    total       NUMERIC(10,2),
    order_ts    TIMESTAMP
) PARTITION BY HASH (cust_id);

-- Создаём четыре партии с modulus=4 и разными remainder
CREATE TABLE orders_p0 PARTITION OF orders
    FOR VALUES WITH (MODULUS 4, REMAINDER 0);

CREATE TABLE orders_p1 PARTITION OF orders
    FOR VALUES WITH (MODULUS 4, REMAINDER 1);

CREATE TABLE orders_p2 PARTITION OF orders
    FOR VALUES WITH (MODULUS 4, REMAINDER 2);

CREATE TABLE orders_p3 PARTITION OF orders
    FOR VALUES WITH (MODULUS 4, REMAINDER 3);

```

- Также существует **Default partition** (дефолтная партиция), она хранит всё то, что не попадает по условию ни в одну из других созданных партиций.

```

CREATE TABLE events_default PARTITION OF events
    DEFAULT;

```

Дефолтная партиция позволяет:

- избегать ошибок при неожиданных значениях;
- собирать «лишние» или редко встречающиеся данные в одном месте;
- упрощать миграцию и реорганизацию схемы: новые диапазоны можно добавить позже, а промежуточные данные будут аккумулироваться в **DEFAULT**.

Патриции можно изменять и удалять.

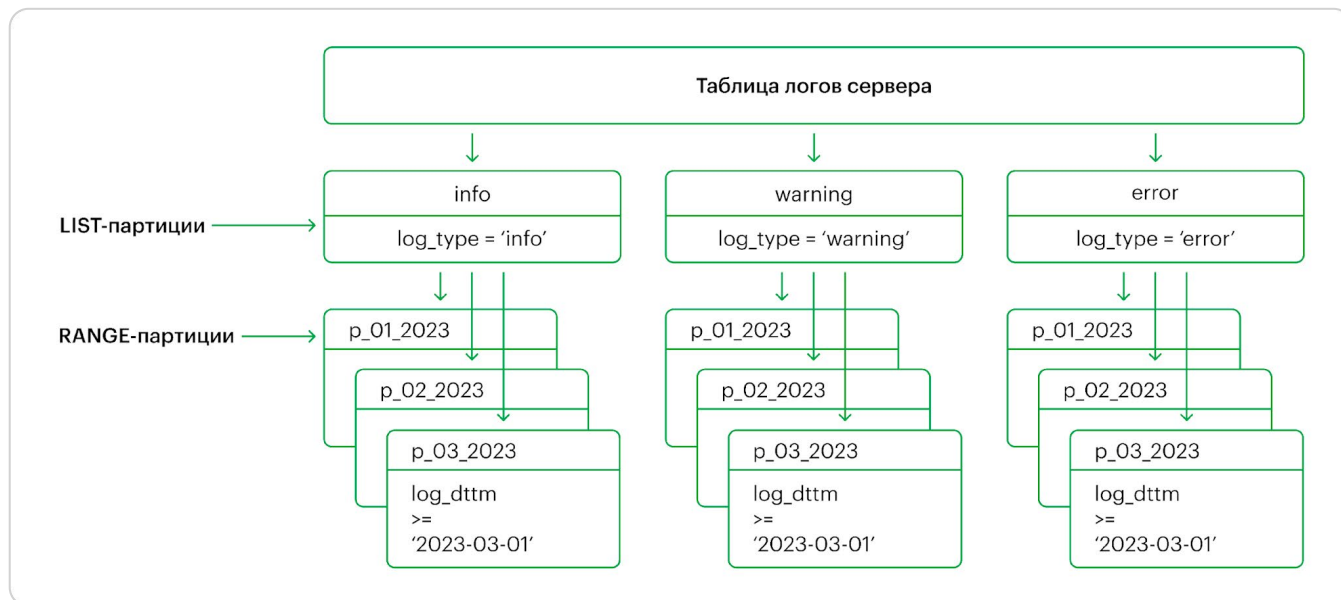
```
alter table table_name add partition p2 start  
( '2026-01-01'::date) end ( '2027-01-01'::date);  
alter table table_name drop partition for ( 'type1');
```

Также можно задать многоуровневое партиционирование.

```
partition by list(list_column)  
  subpartition by RANGE(date_column)  
(  
  partition type1 values ( 'type1')  
  (  
    subpartition p_old start ( '2025-01-01 00:00:00')  
    INCLUSIVE end ( '2026-01-01 00:00:00') EXCLUSIVE,  
    default subpartition def_p  
  ),  
  partition type2 values ( 'type2')  
  (  
    subpartition p_old start ( '2025-01-01 00:00:00')  
    INCLUSIVE end ( '2026-01-01 00:00:00') EXCLUSIVE,  
    default subpartition def_p  
  ),  
  default partition def_p  
  (  
    subpartition p_old start ( '2025-01-01 00:00:00')  
    INCLUSIVE end ( '2026-01-01 00:00:00') EXCLUSIVE,  
    default subpartition def_p  
  )  
);
```

Также можно задать **многоуровневое партиционирование** (Multi-Level Partitioning), которое позволяет разбивать большие таблицы на вложенные уровни партий по разным критериям (например, по диапазону дат и по хеш-значению), что повышает эффективность хранения и выполнения запросов. Это достигается путём создания вложенных подтаблиц.

1. Корневая таблица-партиция (**PARENT**).
2. Первый уровень разбивки (**PARTITION BY ...**).
3. Второй и последующие уровни (каждая под-партиция может сама являться родителем для следующего уровня).



Multi-Level Partitioning


```

CREATE TABLE sales (
    sale_id      SERIAL,
    sale_date    DATE      NOT NULL,
    region       TEXT      NOT NULL,
    amount       NUMERIC
) PARTITION BY RANGE (sale_date);

-- Уровень 1: разбивка по годам
CREATE TABLE sales_2023 PARTITION OF sales
    FOR VALUES FROM ('2023-01-01') TO ('2024-01-01')
    PARTITION BY LIST (region);

CREATE TABLE sales_2024 PARTITION OF sales
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01')
    PARTITION BY LIST (region);

-- Уровень 2: внутри каждого года разбиение по регионам
CREATE TABLE sales_2023_europe PARTITION OF sales_2023
    FOR VALUES IN ('EU');

CREATE TABLE sales_2023_americas PARTITION OF sales_2023
    FOR VALUES IN ('US', 'CA', 'BR');

CREATE TABLE sales_2024_asia PARTITION OF sales_2024
    FOR VALUES IN ('CN', 'IN', 'JP');

```

Рекомендуется использовать максимум 2 уровня партиционирования, чтобы планировщик не строил план запроса слишком долго и проще осуществлялось обслуживание партиций (добавление/удаление новых партиций, очистка дефолтных и т. д.).

Общие рекомендации

В данном разделе описываются общие рекомендации по написанию запросов. Следование данным рекомендациям позволит в большинстве случаев избежать ошибок и проблем.

- Перед запуском запроса используй **EXPLAIN**.
- Если запрос регулярный и работает долго, проанализируй его план с помощью **EXPLAIN ANALYZE**.
- Если таблица партиционирована, используй **условие на поле партиционирования**.
- Если в таблице есть **индексы**, то старайся использовать условие на поле с индексом.
- Для больших запросов лучше создавать временные промежуточные таблицы, чтобы переиспользовать одни и те же данные.
- Не использовать **OR, IN, LIKE** в условиях соединения по полям в операциях **JOIN**.
- Перед операцией **JOIN** необходимо по максимуму уменьшать количество данных с помощью **применения фильтров**.
- Избегай **SELECT ***, явно указывай нужные столбцы.
- Избегай операции **ORDER BY**. Отсортировать значения можно на финальных стадиях запроса либо в IDE (DBeaver).
- Избегай чрезмерной дедубликации через **DISTINCT / GROUP BY**, так как эти операции являются тяжёлыми.
- Избегай операции **UNION**, вместо неё используй операцию **UNION ALL**. Первая, помимо объединения данных, ещё и дедублицирует записи, во время данного процесса запускается сортировка, которую нужно также избегать.
- Избегай **подзапросов**, используй вместо этого **JOIN**.

Частые ошибки при составлении запросов

ИСПОЛЬЗОВАНИЕ OR В УСЛОВИЯХ ФИЛЬТРАЦИИ

Условия с OR могут предотвратить использование индексов и партиций, приводя к полному сканированию таблицы.

ПРИМЕР

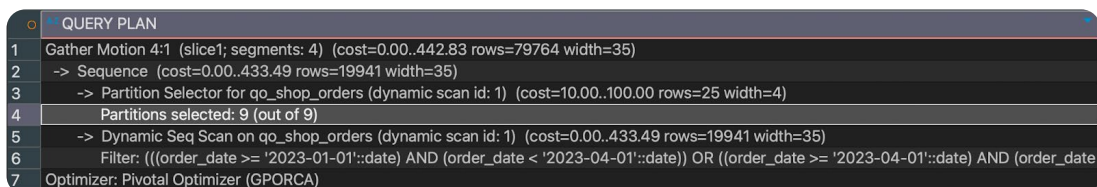
```
SELECT *  
FROM my_table  
WHERE col1 = 'A' OR col2 = 'B';
```

Перепиши запрос с использованием **UNION ALL**.

```
SELECT * FROM my_table WHERE col1 = 'A'  
  
UNION ALL  
  
SELECT * FROM my_table WHERE col2 = 'B';
```

ПРИМЕР 2

```
explain  
select *  
from databases_and_sql.qo_shop_orders  
where order_date >= '2023-01-01' and order_date <  
    '2023-04-01'  
    or order_date >= '2023-04-01' and order_date <  
    '2023-07-01'
```



```
QUERY PLAN  
1  Gather Motion 4:1 (slice1; segments: 4) (cost=0.00..442.83 rows=79764 width=35)  
2  -> Sequence (cost=0.00..433.49 rows=19941 width=35)  
3  -> Partition Selector for qo_shop_orders (dynamic scan id: 1) (cost=10.00..100.00 rows=25 width=4)  
4  Partitions selected: 9 (out of 9)  
5  -> Dynamic Seq Scan on qo_shop_orders (dynamic scan id: 1) (cost=0.00..433.49 rows=19941 width=35)  
6  Filter: (((order_date >= '2023-01-01'::date) AND (order_date < '2023-04-01'::date)) OR ((order_date >= '2023-04-01'::date) AND (order_date  
7  Optimizer: Pivotal Optimizer (GPORCA)
```

Таблица партиционирована по полю **order_date** по квартално.
На первый взгляд условия в запросе написаны корректно, однако,
судя по плану запроса, сканируются все 9 партиций.

Перепишем запроса используя «**union all**» вместо «**or**».

```
explain
select *
from databases_and_sql.qo_shop_orders
where order_date >= '2023-01-01' and order_date <
'2023-04-01'
union all
select *
from databases_and_sql.qo_shop_orders
where order_date >= '2023-04-01' and order_date <
'2023-07-01'
```

QUERY PLAN	
1	Gather Motion 4:1 (slice1; segments: 4) (cost=0.00..863.87 rows=11905 width=35)
2	-> Append (cost=0.00..862.48 rows=2977 width=35)
3	-> Sequence (cost=0.00..431.17 rows=1395 width=35)
4	-> Partition Selector for qo_shop_orders (dynamic scan id: 1) (cost=10.00..100.00 rows=25 width=4)
5	Partitions selected: 2 (out of 9)
6	-> Dynamic Seq Scan on qo_shop_orders (dynamic scan id: 1) (cost=0.00..431.17 rows=1395 width=35)
7	Filter: ((order_date >= '2023-01-01'::date) AND (order_date < '2023-04-01'::date))
8	-> Sequence (cost=0.00..431.20 rows=1582 width=35)
9	-> Partition Selector for qo_shop_orders (dynamic scan id: 2) (cost=10.00..100.00 rows=25 width=4)
10	Partitions selected: 2 (out of 9)
11	-> Dynamic Seq Scan on qo_shop_orders qo_shop_orders_1 (dynamic scan id: 2) (cost=0.00..431.20 rows=1582 width=35)
12	Filter: ((order_date >= '2023-04-01'::date) AND (order_date < '2023-07-01'::date))
13	Optimizer: Pivotal Optimizer (GPORCA)

Теперь по плану запроса видно, что в каждом подзапросе сканируется по 2 партии (одна с нужными датами, вторая дефолтная).

ФУНКЦИИ В УСЛОВИЯХ JOIN

Использование функций вроде `upper()` или `substring()` в условиях `JOIN` может привести к тому, что оптимизатор PostgreSQL не сможет эффективно задействовать индексы и механизм партиционирования.

ПРИМЕР

```
SELECT *
FROM table1 t1
JOIN table2 t2
ON t1.col = upper(t2.col);
```

Как исправить

Предварительно вычислай значения в отдельном столбце.

```
CREATE TEMP TABLE tmp AS
SELECT *, upper(col) AS upper_col
FROM table2;

SELECT *
FROM table1 t1
JOIN tmp t2
ON t1.col = t2.upper_col;
```

ИЛИ

```
ALTER TABLE table2 ADD COLUMN upper_col VARCHAR(100);
UPDATE table2 SET upper_col = UPPER(col);

SELECT *
FROM table1 t1
JOIN table2 t2
ON t1.col = t2.upper_col;
```

ФУНКЦИИ В УСЛОВИЯХ WHERE

Аналогично **JOIN**, применение любых функций к полям, участвующим в партиционировании или индексировании, может привести к тому, что оптимизатор PostgreSQL перестанет учитывать такие индексы и партиции.

ПРИМЕР	<pre>SELECT * FROM table1 t1 WHERE LEFT(phone_number, 2) = '+7'</pre>
	Как исправить По возможности стоит переписывать условия без функций, используя альтернативные SQL-подходы.
	<pre>SELECT * FROM table1 t1 WHERE phone_number LIKE '+7%'</pre>
	В этом случае оптимизатор сможет использовать индекс (если он существует). Однако важно учитывать, что LIKE с шаблоном вида '%...%' по сути даёт тот же эффект, что и функции — индекс не будет применён. Оптимальный вариант — когда символ % стоит только в конце строки.

ИСПОЛЬЗОВАНИЕ IN И АЛЬТЕРНАТИВЫ

Оператор **IN** может быть неэффективным для больших списков, так как не всегда использует индексы.

ПРИМЕР	<pre>SELECT * FROM my_table WHERE id IN (SELECT id FROM my_table2);</pre>
	Как исправить Для больших списков используй JOIN с временной таблицей.
	<pre>SELECT * FROM my_table t1 INNER JOIN my_table2 t2 ON t1.id = t2.id;</pre>

НЕОПТИМАЛЬНЫЕ ПОДЗАПРОСЫ

Коррелированные подзапросы могут быть медленными, так как выполняются для каждой строки внешнего запроса.

ПРИМЕР

```
SELECT *
FROM orders o
WHERE EXISTS (
    SELECT 1
    FROM customers c
    WHERE c.id = o.customer_id AND c.status = 'active'
);
```

Как исправить

Заменяй подзапросы на JOIN.

```
SELECT o.*
FROM orders o
    JOIN customers c
        ON o.customer_id = c.id
WHERE c.status = 'active';
```

Заключение

Оптимизация запросов в PostgreSQL — это процесс, требующий глубокого понимания архитектуры базы данных и внимательного подхода к написанию SQL-запросов.

Тестирование и мониторинг производительности запросов — ключ к поддержанию высокой эффективности системы. Используй **EXPLAIN** и **EXPLAIN ANALYZE** и следуй лучшим практикам, чтобы твои запросы работали быстро и надёжно.